How the orchestrator works

Commerce AI Assistant (store-assistant) · reference notes · 30 October 2026

The orchestrator does several things in sequence, so this guide splits it into two flowcharts: the overall path a chat message takes, then a closer look at the agent loop where Claude is called. A worked example, a map of the code, and a comparison of the Claude side and the tools side follow.

1. The path every chat message takes

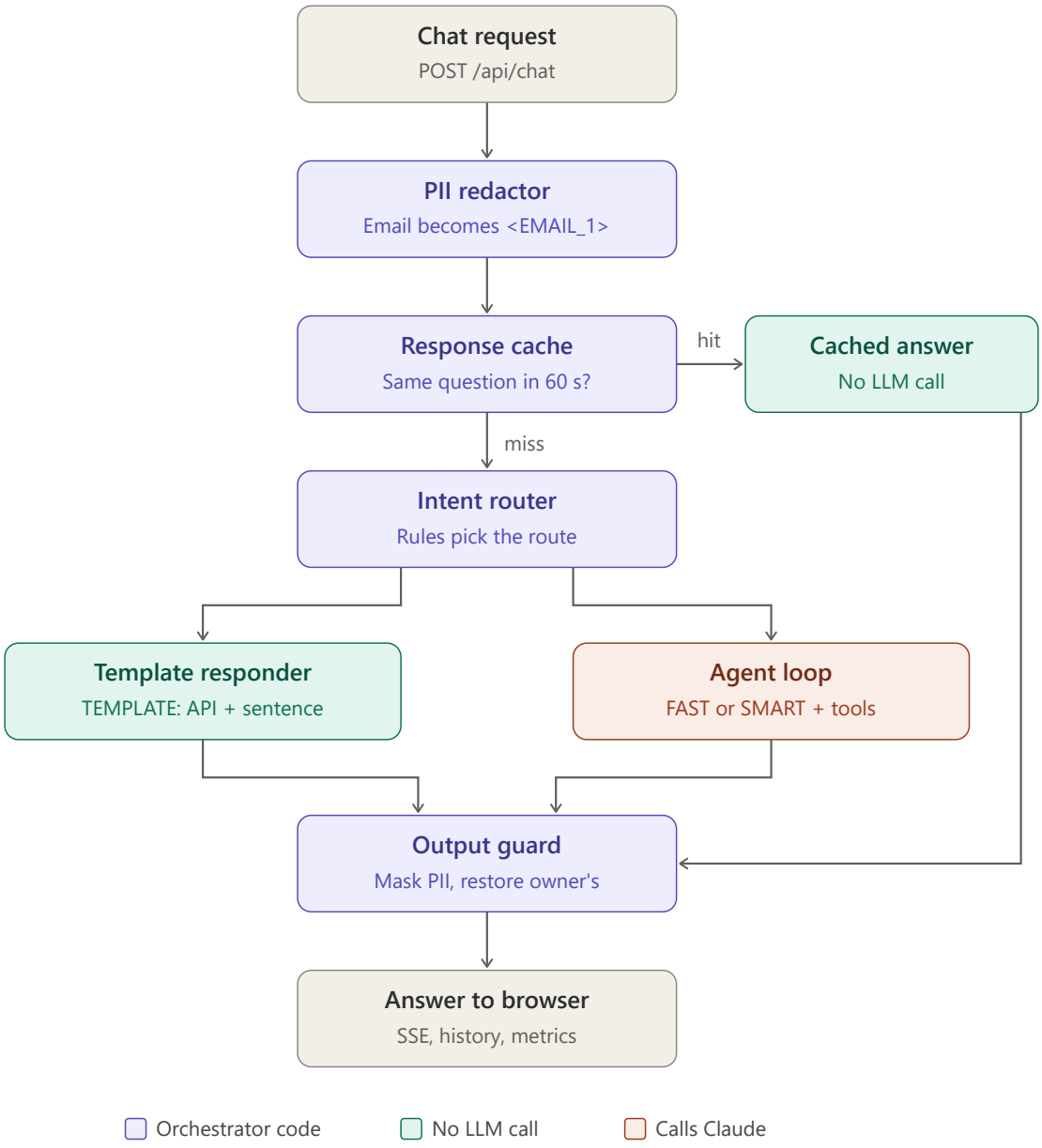


Figure 1. Request pipeline in `AgentService.handle()`

How the main path works

- **Redaction:** this always happens first, so emails, phone numbers and cards are replaced before anything else sees the text. That includes the cache key and Claude.

- **Cache:** a cache hit skips everything up to the output guard.
- **Router:** this is simple pattern matching, with no AI involved. An order ID plus a status word ("where is ORD-1001") goes to the template. Words like cancel, refund, charged or damaged go to SMART (Sonnet). Everything else goes to FAST (Haiku).
- **Template fallback:** if the template can't answer, for example because the order isn't found in a way it can phrase, the request falls through to the agent loop.

2. Inside the agent loop

This is the part that handles FAST and SMART requests.

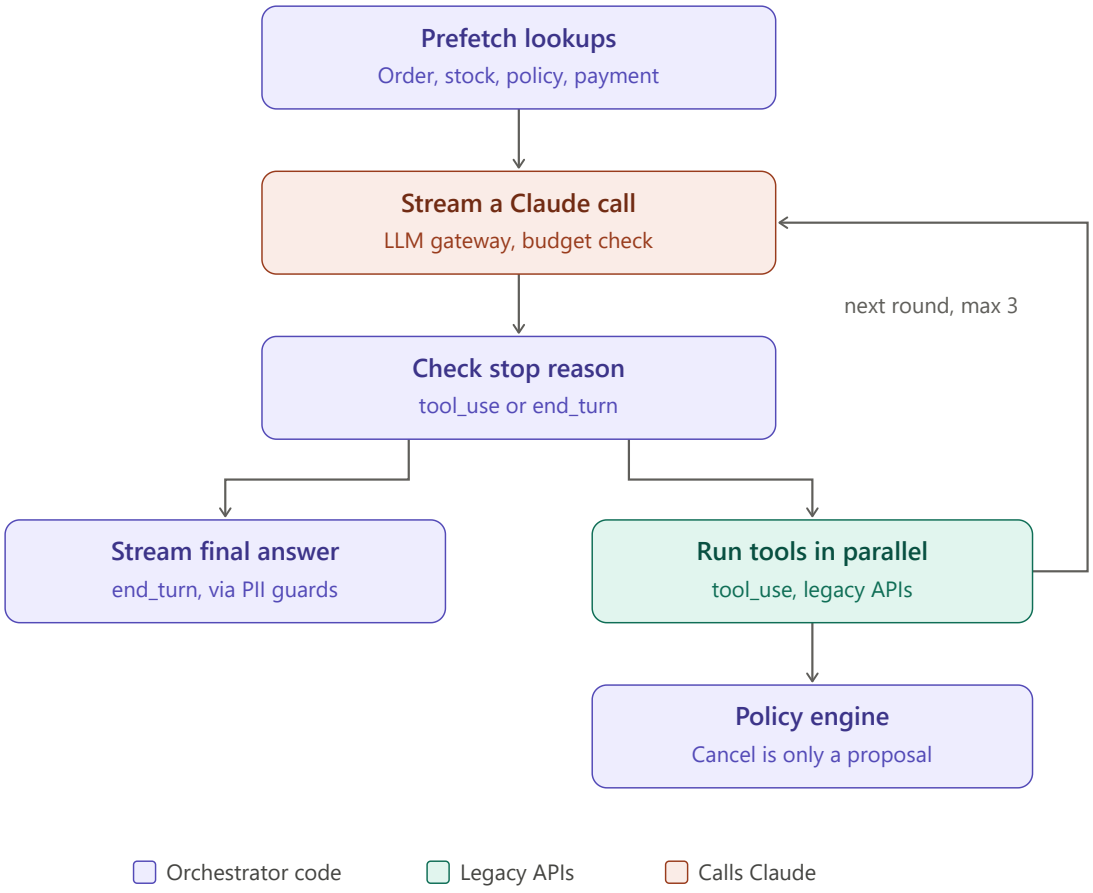


Figure 2. Agent loop in `AgentService.runAgent()`

One question, step by step

As Asha (`u-1001`), you ask "*My payment failed for ORD-1004, what now?*"

1. **Redactor:** there's no PII, so the text is unchanged.
2. **Cache:** it's a new question, so it misses.
3. **Router:** "payment failed" sends it to **SMART** (Sonnet).
4. **Prefetch:** the question contains `ORD-1004` and "payment", so `get_order` and `get_payment_status` run in parallel against the mock APIs before Claude is called. The results are added to the conversation as if Claude had called those tools itself.
5. **Claude, round 1:** Sonnet sees that the payment failed and asks for `create_payment_link`.
6. **Tools:** the orchestrator calls the Payment API with Asha's ID. That ID comes from the request, never from Claude. The result is trimmed and checked for PII again.
7. **Claude, round 2:** it writes the answer with the payment link. Stop reason: `end_turn`.
8. **Streaming guard:** each piece of text is checked for PII before it's sent to the browser.
9. **Finish step:**
    - The output guard checks the full answer.

- The answer is saved to the conversation history.
- It isn't cached, because a payment link is a write action.
- The metrics are updated.
- The browser receives the `answer` event, which carries the route, model, cost and tools used.

Where each step lives in the code

All paths are under `orchestrator/src/main/java/com/example/commerceai/`.

Step	Class
HTTP and SSE	`api/ChatController`
Whole flow	`agent/AgentService.handle()`
Redaction, vault, guards	`privacy/PiiRedactor`, `PiiVault`, `OutputGuard`, `StreamingGuard`
Cache and routing	`routing/ResponseCache`, `IntentRouter`, `PrefetchPlanner`
Template answers	`agent/TemplateResponder`
Claude calls, budget, cost	`llm/LlmGateway`, `ModelPricing`, `UsageLedger`
Tools and legacy APIs	`tools/CommerceTools`, `CommerceApiClient`
Cancel confirmation	`tools/PolicyEngine`, `api/ActionController`

Two safety limits

- **Round limit:** if Claude still wants tools after 3 rounds, the loop stops and the user gets a handoff message instead of an endless loop.
- **Refusals:** if Claude refuses, the streamed text is withdrawn and the same handoff message is shown.

3. Claude calls compared with tools and legacy APIs

These are the two halves of the agent loop. **Claude decides; the tools get the data.** Claude never touches your APIs. It can only ask the orchestrator to call one, and the orchestrator decides whether and how.

	Claude calls (`llm/`)	Tools and legacy APIs (`tools/`)
Role	The brain: understands the question, decides what's needed, writes the answer	The hands: fetch real data and perform actions
Talks to	Claude, on Bedrock or the Claude API	Your existing Order, Inventory and Payment APIs (here, the mock APIs)
Input / output	Text in, text or a tool request out	A tool name with arguments in, JSON data out
Cost	Paid per token (Haiku ≈ $0.0001, Sonnet ≈ $0.004 per request)	No LLM cost. A normal internal API call.
Speed	Slow: about 1–4 s	Fast: about 60 ms each
Predictable?	No. Wording varies, and it can be wrong.	Yes: same input, same result
Main risks	Overspending, prompt injection, refusals	Data leaks, unauthorized actions
Safeguards	Budget cap, model choice per route, token limits, cost metering	Ownership checks, trimmed results, PII re-redaction, confirmation for writes

Claude calls, budget and cost (`llm/`)

`LlmGateway` is the **only** place that talks to Claude. On every call it:

1. **Checks the budget** (`effectiveRoute`). Above 90% of the monthly cap, Sonnet requests drop down to Haiku. At 100%, Claude isn't called at all, and the user gets a polite "assistant is limited" message.
2. **Picks the model** (`modelFor`): Haiku for FAST, Sonnet for SMART.
3. **Streams the response** and assembles the final message.
4. **Meters the cost.** `ModelPricing` turns token counts into dollars. Cached prompt tokens are 90% cheaper, and writing to the cache costs 25% more. `UsageLedger` adds it all up for `/api/metrics` .

Because every Claude call goes through this one class, there's a single place to enforce spend limits and to switch providers (Claude API or Bedrock).

Tools and legacy APIs (`tools/`)

`CommerceTools` defines the 7 tools Claude may ask for:

Read-only	Writes (go through the policy engine)
`get_order` , `list_orders` , `check_inventory` , `get_payment_status` , `search_policy`	`cancel_order` (proposal only), `create_payment_link`

When Claude asks for a tool, `CommerceTools.execute()` :

1. **Uses the logged-in user's ID**, never an ID Claude supplies. Even if a prompt injection tells Claude to "show user u-1002's orders", the API call still carries Asha's ID, so the API returns "not found".

2. **Calls the legacy API** through `CommerceApiClient`, which sends HTTP requests to the Order, Inventory and Payment services with an 800 ms timeout.
3. **Trims the response** to the fields Claude needs, such as status, ETA and items. Names, addresses and card details are dropped, which also keeps requests small and cheap.
4. **Redacts PII again** in case an API returned some anyway.
5. **Routes write actions through the policy engine.** `cancel_order` only creates a proposal. The order changes only when the user clicks Confirm, and that call uses an idempotency key so a double click doesn't cancel twice.

How they work together, with costs

"Is SKU-HEADPHONES in stock?"

```
Prefetch    → check_inventory("SKU-HEADPHONES")      tools/   ~60 ms   $0
Claude call → reads {"available": 42}, writes answer  llm/    ~1.2 s   ~$0.0001 (Haiku)
```

"Cancel ORD-1002"

```
Prefetch      → get_order("ORD-1002") → status "placed"          tools/  $0
Claude call 1 → asks for cancel_order("ORD-1002")                llm/    ~$0.003 (Sonnet)
Tool          → policy engine: placed = cancellable → proposal   tools/  $0, nothing changed yet
Claude call 2 → "I've prepared the cancellation, please confirm" llm/    ~$0.002
User clicks Confirm → Order API cancels (idempotency key)        tools/  $0, no Claude involved
```

> Almost all of the cost and most of the wait comes from the Claude calls. That's why the design minimizes them: templates, the cache, prefetching, and Haiku over Sonnet. The tools are cheap and fast; their job is safety, making sure Claude can only see and do what the logged-in user is allowed to.